

Instructions: Please answer the following questions, making sure to write legibly on the following pages. The code below is a subsection of a Python file that runs Flask for a library management system.

- Since we are writing Python code, please be careful about spacing/indentation. Leave enough room that it is obvious when you want things indented.
- Make sure to write legibly.
- All questions are worth equal points unless otherwise stated.
- The quiz is cumulative and may include material from previous weeks.

```
from flask import Flask, request, jsonify

app = Flask(__name__)

@app.route('/api/books/<book_id>', methods=['GET'])
def get_book(book_id):

    print(f"{book_id}")
    print(f"{request.args.to_dict()}")
    print(f"{request.method}")

    return jsonify({
        'book_id': book_id,
        'args': request.args.to_dict(),
        'method': request.method
    }), 200

@app.route('/api/books/search/<search_term>', methods=['GET'])
def search_books(search_term):
    results = get_search_results(search_term)
    # INSERT ANSWER HERE

@app.route('/api/books', methods=['POST'])
def create_book():
    data = request.get_json()
    title = data.get('title')
    author = data.get('author')
    return jsonify({'message': f'Created book: {title} by {author}'}), 201
```

1. An HTTP request consists of several parts. Please identify where we would expect each of the following types of information to be located in an HTTP request:

Information Type	Part of HTTP Request
Search Terms from a google query	
Information submitted for inserting data	
Authentication token for API access	

2. Consider the `get_book` function shown on the first page. If a client makes a GET request to the following URL:

`GET /api/books/12345?format=json&include_reviews=true`

What would be printed to the console? Please write the exact output that would appear.

Consider the `search_books` function shown on the first page. The function `get_search_results` takes a search term and returns a list of tuples, where each tuple contains a book title and its relevance score (higher is better). For example, the function `get_search_results("monster books")` might return:

```
[
    ('Frankenstein', 0.25),
    ('Dracula', 0.75),
    ('The Strange Case of Dr. Jekyll and Mr. Hyde', 0.45),
    ('The Island of Dr. Moreau', 0.90)
]
```

The results are **NOT ordered** by relevance score, so higher relevance scores can appear before or after lower ones.

3. Complete the `search_books` function by writing code at `# INSERT ANSWER HERE`. Your code should:

- Filter the results to only include books with a relevance score greater than 0.5
- Create a list containing only the titles (not the scores) of the filtered books
- If there are results, return them using `jsonify({'results': <list_of_titles>})` with status code 200
- If there are no relevant results return `jsonify({'results': []})` with status code 200

4. Write code that finds and returns the first result from `get_search_results(search_term)` that has a relevance score greater than 1.0. Your code should:

- Return the *first* result (book title only, not the score) that has a relevance score greater than 1.0
- If a result is found, return them using `jsonify({'book': <title>})` with status code 200
- If there are no results with relevance greater than 1.0, return `jsonify({'error': 'No results found'})` with status code 404